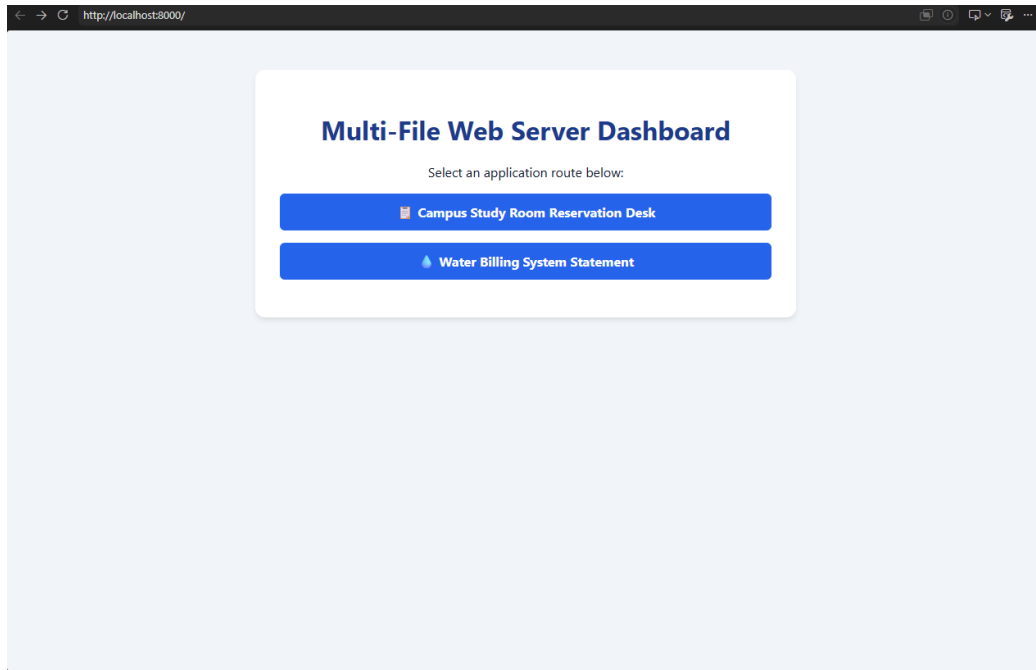
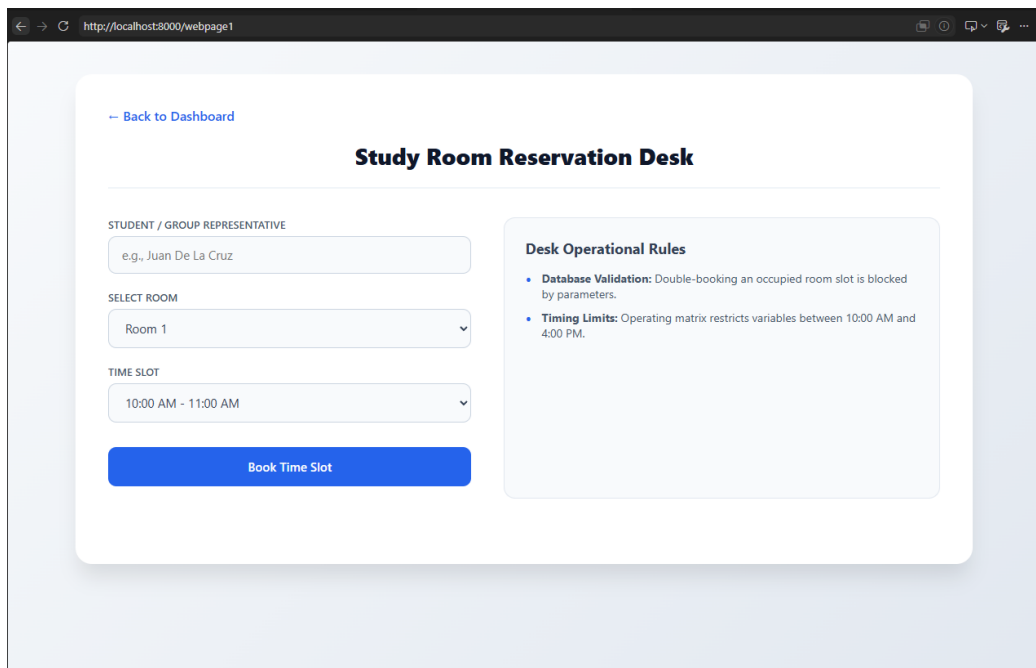


Screenshots:

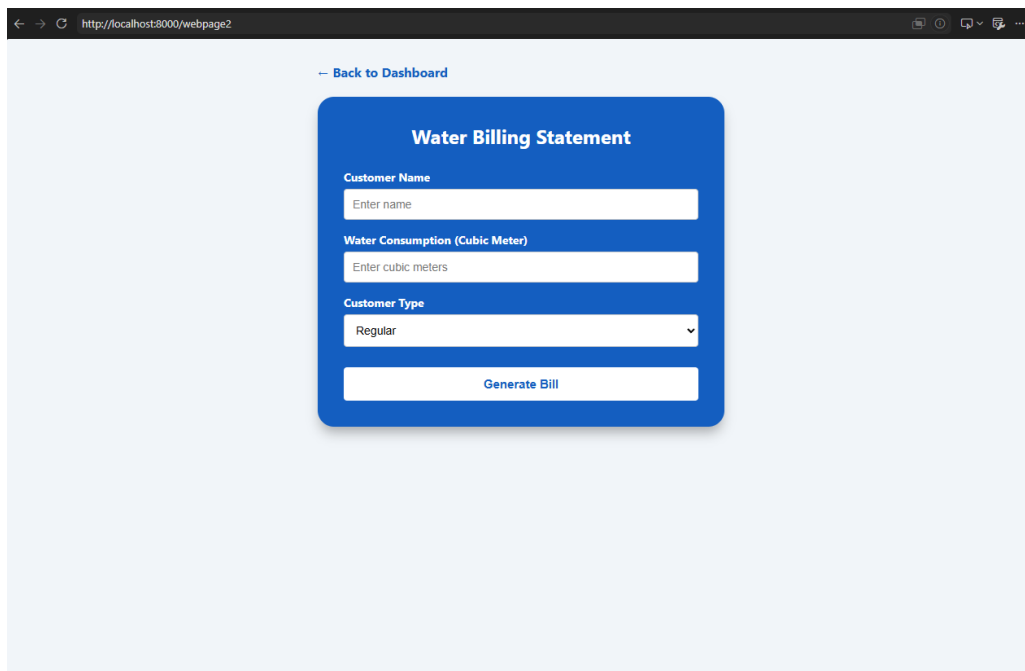
multiFile.js:



WebPage1:



WebPage2:



The screenshot shows a web browser window with the address bar displaying `http://localhost:8000/webpage2`. The page content features a blue button labeled `← Back to Dashboard` at the top. Below it is a blue-bordered box titled **Water Billing Statement**. Inside this box, there are three input fields: **Customer Name** with the placeholder text 'Enter name', **Water Consumption (Cubic Meter)** with the placeholder text 'Enter cubic meters', and **Customer Type** with a dropdown menu currently showing 'Regular'. At the bottom of the box is a white button labeled `Generate Bill`.

CMD.EXE Command

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\ITPE\Fulgencio_FinalEA2> cmd.exe
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

D:\ITPE\Fulgencio_FinalEA2>
```

NPM INSTALL Command

```
D:\ITPE\Fulgencio_FinalEA2>npm install -g npm

added 1 package in 8s

15 packages are looking for funding
  run `npm fund` for details

D:\ITPE\Fulgencio_FinalEA2>
```

NPM INIT Command

```
D:\ITPE\Fulgencio_FinalEA2>npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See `npm help init` for definitive documentation on these fields and exactly what they do.

Use `npm install <pkg>` afterwards to install a package and save it as a dependency in the package.json file.

Press ^C at any time to quit.
package name: (fulgencio_finalea2)
version: (1.0.0)
description:
entry point: (multiFile.js)
test command:
git repository:
keywords:
author:
license:
type: (commonjs)
About to write to D:\ITPE\Fulgencio_FinalEA2\package.json:

{
  "name": "fulgencio_finalea2",
  "version": "1.0.0",
  "description": "",
  "main": "multiFile.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "type": "commonjs"
}

Is this OK? (yes)
```

NPM INSTALL EXPRESS Command

```
D:\ITPE\Fulgencio_FinalEA2>npm install express

added 67 packages, and audited 68 packages in 4s

25 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

RUNNING multiFile.js Command

```
D:\ITPE\Fulgencio_FinalEA2>node multiFile.js
Server is running on http://localhost:8000
□
```

Reflection:

1. What did you learn from creating and running the multi-file Express.js web server?

I learned how to break down a large, complex backend script into smaller modular parts by using `require` and `module.exports` to handle communication between separate files. This process helped me understand how web routes map directly to external module functions instead of stuffing all request logic into a single file.

2. What challenges or errors did you encounter during the activity? How did you solve them?

I encountered path errors where the main server file could not find the module files inside the directory. I solved this by verifying my folder structure, placing the scripts inside the public folder, and ensuring `module.exports` were declared correctly at the bottom of each file.

3. How did separating the server into multiple files improve the organization and maintainability of your project?

It cleans up the codebase by implementing separation of concerns, which ensures that each individual file has exactly one job to perform. If a bug occurs on the water billing page, I can debug `webpage2.js` directly without risking breaking the main server file or other working pages.

4. What new skills or concepts did you gain about Node.js, Express.js, modules, routing, or project structure?

I gained skills in setting up a backend server from scratch using `npm init`, installing dependencies, and configuring paths using the Express static file middleware. I also mastered the logic of modular routing, which allows a server to scale up efficiently as more endpoints are added.

5. How can you apply what you learned in future web development or capstone projects?

I will use this modular structure in our capstone project by breaking our frontend down into small, isolated React components and separating our PHP backend script logic into clean includes. This keeps our code highly organized, prevents conflicts when multiple team members are coding at the same time, and makes the system much easier to expand.